

TD vols commerciaux : éléments de correction

Soit **Structure Classe**

```
{  
    champ code : entier  
    champ nom : chaîne de caractères  
    champ prix : réel  
}
```

Soit **Structure Passager**

```
{  
    champ numero : entier  
    champ nom : chaîne de caractères  
    champ prenom : chaîne de caractères  
    champ codeClasse : chaîne de caractères  
}
```

Soit **Structure Bagage**

```
{  
    champ numero : entier  
    champ poids : entier  
    champ numBillet : entier  
}
```

Fonction : **ChargerListeClasse**

Entrée : - numero : entier

Local : - chemin : chaîne de caractères
- details : tableau de chaînes de caractères
- flux : flux de lecture

Sortie : - listeClasses : liste de **structure Classe**

Début

Soit listeClasses = liste de **structure Classe**

Soit chemin = "C" + numero + ".txt".

Si chemin existe **Alors**

Soit flux = **OuvrirFluxLecture**(chemin)

Soit ligne = ""

Tant que (Soit ligne = **LireProchaineLigne**(flux)) est non **NULL** **Faire**

Soit details = **Séparer**(ligne, ';')

Soit c = Classe {details[0], details[1], **Réel**(details[2])}

Ajouter(listeClasses, c)

Fin Tant Que

FermerFluxLecture(flux)

Fin Si

Retourner listeClasses

Fin

Fonction : **ChargerListePassager**

Entrée : - numero : entier

Local : - chemin : chaîne de caractères
- details : tableau de chaînes de caractères
- flux : flux de lecture

Sortie : - listePassagers : liste de structure Passager

Début

Soit listePassagers = liste de structure Passager

Soit chemin = "P" + numero + ".txt".

Si chemin existe Alors

 Soit flux = OuvrirFluxLecture(chemin)

 Soit ligne = ""

 Tant que (Soit ligne = LireProchaineLigne(flux)) est non NULL Faire

 Soit details = Séparer(ligne, ';')

 Soit Passager p = Passager {Entier(details[0]), details[1], details[2], details[3]}

 Ajouter(listePassagers, p)

 Fin Tant Que

 FermerFluxLecture(flux)

Fin Si

Retourner listePassagers

Fin

Fonction : **ChargerListeBagage**

Entrée : - numero : entier

Local : - chemin : chaîne de caractères
- details : tableau de chaînes de caractères
- flux : flux de lecture

Sortie : - listeBagages : liste de structure Bagage

Début

Soit listeBagage = liste de structure Bagage

Soit chemin = "P" + numero + ".txt".

Si chemin existe Alors

 Soit flux = OuvrirFluxLecture(chemin)

 Soit ligne = ""

 Tant que (Soit ligne = LireProchaineLigne(flux)) est non NULL Faire

 Soit details = Séparer(ligne, ';')

 Soit Bagage b = Bagage {Entier(details[0]), Entier(details[1]), Entier(details[2])}

 Ajouter(listeBagage, b)

 Fin Tant Que

 FermerFluxLecture(flux)

Fin Si

Retourner listeBagage

Fin

Fonction : **RechercherPassager**

Entrée : - numero : entier

- listePassagers : liste de structure Passager

Local : - i : entier

Sortie : - passager : structure Passager ou NULL

Début

Soit i = 0

Tant Que i < Longueur(listePassagers) Faire

 Si listePassagers[i].numero == numero Alors

 Retourner listePassagers[i]

 Fin Si

 Soit i = i + 1

Fin Tant Que

Retourner NULL

Fin

Fonction : **RechercherClasse**

Entrée :
- code : chaîne de caractères
- listeClasses : liste de **structure Classe**

Local :
- i : entier

Sortie :
- classe : **structure Classe** ou **NULL**

Début

```
Soit i = 0
Tant Que i < Longueur(listeClasses) Faire
|   Si listeClasses[i].code est égal à code Alors
|   |   Retourner listeClasses[i]
|   Fin Si
|   Soit i = i + 1
Fin Tant Que
Retourner NULL
```

Fin

Fonction : **RechercherPrixDepuisNumeroBillet**

Entrée :
- listePassagers : liste de **structure Passager**
- listeClasses : liste de **structure Classe**
- numPassager : entier

Local :
- p : **structure Passager**
- c : **structure Classe**

Sortie :
void

Début

```
Soit p = RechercherPassager(numPassager, listePassagers)
Si p est non NULL Alors
|   Soit c = RechercherClasse(p.codeClasse, listeClasses)
|   Si c est non NULL Alors
|   |   Afficher("Prix du billet du passager : {c.prix} euros.")
|   Sinon
|   |   Afficher("Classe introuvable")
|   Fin Si
Sinon
|   Afficher("Passager introuvable")
Fin Si
```

Fin

Fonction : **RechercherBagageDepuisPassager**

Entrée :
- listeBagages : liste de **structure Bagage**
- passager : **structure Passager**

Local :
- i : entier

Sortie :
- bagages : liste de **structure Bagage**

Début

```
Soit bagages = liste de structure Bagage
Pour Chaque structure Bagage b dans listeBagages Faire
|   Si b.numBillet == passager.numero Alors
|   |   Ajouter(bagages, b)
|   Fin Si
Fin Pour Chaque
Retourner bagages
```

Fin

Procédure : **RechercherPrixDepuisNumeroBillet**

Entrée :
- listePassagers : **liste** de **structure Passager**
- listeClasses : **liste** de **structure Classe**
- listeBagages : **liste** de **structure Bagage**

Local :
- i : **entier**
- c : **structure Classe**
- bagages : **liste** de **structure Bagage**
- sommePoids : **entier**

Sortie : **void**

Début

```
Afficher("Numéro Passager Classe Nombre de bagages Poids cumulé")
Pour Chaque structure Passager p dans listePassagers Faire
|   Soit c = RechercherClasse(p.codeClasse, listeClasses)
|   Si c est non NULL Alors
|   |   Soit listeBagages = RechercherBagagesDepuisPassager(listeBagages, p)
|   |   Soit sommePoids = 0
|   |   Pour Chaque structure Bagage b dans listeBagages Faire
|   |   |   Soit sommePoids = sommePoids + b.poids
|   |   Fin Pour Chaque
|   Fin Si
|   Afficher("{"p.numero}" {"p.nom}" {"p.prenom}" {"Longueur(listeBagages)} {"sommePoids}")
Fin Pour Chaque
```

Fin

Fonction : **CalculerPrix**

Entrée :
- listeBagages : **liste** de **structure Bagage**
- c : **structure Classe**

Local :
- bagages : **liste** de **structure Bagage**
- sommePoids : **entier**

Sortie :
- prix : **réel**

Début

```
Soit prix = c.prix
Soit sommePoids = 0
Pour Chaque structure Bagage b dans listeBagages Faire
|   Soit sommePoids = sommePoids + b.poids
Fin Pour Chaque
Si sommePoids > 20 ET sommePoids ≤ 30 Alors
|   Soit prix = prix * 1.1
Sinon Si sommePoids ≥ 31 ET sommePrix ≤ 40 Alors
|   Soit prix = prix * 1.2
Sinon Si sommePoids ≥ 41 Alors
|   Soit prix = prix * 1.3
Fin Si
Retourner prix
```

Fin

Procédure : **AfficherChiffreAffairesVol**

Entrée :
- listePassagers : liste de structure Passager
- listeClasses : liste de structure Classe
- listeBagages : liste de structure Bagage

Local :
- caTotal, caPassager : réel
- c : structure Classe
- bagages : liste de structure Bagage

Sortie :
void

Début

```
Pour Chaque structure Passager p dans listePassagers Faire
    Soit c = RechercherClasse(p.codeClasse, listeClasses)
    Si c est non NULL Alors
        Soit listeBagages = RechercherBagagesDepuisPassager(listeBagages, p)
        Soit caPassager = CalculerPrix(bagages, c)
        Soit caTotal = caTotal + caPassager
        Afficher("CA pour le passager {p.numero} : {caPassager} euros.")
    Fin Si
Fin Pour Chaque
Afficher("CA pour le vol : {caTotal} euros.")
```

Fin

Procédure : **Main**

Entrée :
void

Local :
- numVol : entier
- listePassagers : liste de structure Passager
- listeClasses : liste de structure Classe
- listeBagages : liste de structure Bagage
- choix : entier
- quitter : booléen

Sortie :
void

Début

```
Soit numVol = SaisirEntier()
Soit listePassagers = ChargerListePassager(numVol)
Soit listeClasses = ChargerListeClasse(numVol)
Soit listeBagages = ChargerListeBagage(numVol)
Soit quitter = FAUX
Soit choix = 0
Tant Que quitter est FAUX Faire
    Soit choix = 0
    AfficherMenu()
    Tant Que choix < 1 OU choix > 4 Faire
        Soit choix = SaisirEntier()
        Selon Cas choix Faire
            Cas 1 :
                Soit np = SaisirEntier()
                RechercherPrixDepuisNumeroBillet(listePassagers, listeClasses, np)
            Cas 2 :
                AfficherPassagers(listePassagers, listeClasses, listeBagages)
            Cas 3 :
                AfficherChiffreAffaireVol(listePassagers, listeBagages, listeClasses)
            Cas 4 :
                Soit quitter = VRAI
        Fin Selon Cas
    Fin Tant Que
Fin Tant Que
```

Fin